

Swarm

Arturo Silvelo

Try New Roads

Swarm

Swarm

Docker Swarm es la **solución nativa de orquestación** de Docker que permite gestionar un cluster de motores Docker como un **sistema virtual único**.

Elementos:

- **Nodo:** máquina que participa en el clúster.
- **Servicio:** Contenedor y réplicas.
- **Stack:** conjunto de servicios desplegados y gestionados juntos.

Características principales

- **Gestión de cluster integrada** - sin software adicional
- **Escalado automático** - ajusta réplicas según demanda
- **Reconciliación del estado deseado** - auto-reparación
- **Service discovery + Balanceo de carga** - DNS y distribución automática
- **Rolling updates** - despliegues sin downtime con rollback
- **Seguro por defecto** - TLS

Gestión Swarm

Estructura Swarm

Un swarm se compone de nodos manager y nodos worker. Los managers son responsables de gestionar el estado del cluster, tomar decisiones de orquestación y proporcionar las APIs del swarm - siempre debe haber al menos uno y se recomienda un número impar para garantizar el consenso. Los workers ejecutan las tareas (contenedores) que les asignan los managers.

Añadir al swarm

Cuando creas un swarm, inicias con un solo Docker Engine en modo Swarm. Para aprovechar todas las ventajas de Swarm, puedes añadir más nodos:

- **Añadir nodos worker** aumenta la capacidad del cluster. Los servicios se distribuyen entre todos los nodos disponibles, sean workers o managers, permitiendo escalar el swarm sin afectar el consenso de los managers.
- **Añadir nodos manager** incrementa la tolerancia a fallos. Los managers gestionan la orquestación y el estado del cluster. Entre ellos, uno actúa como líder. Si el líder falla, los managers restantes eligen un nuevo líder y el cluster sigue funcionando.

- Crear red para el clúster

```
docker network create swarm-network
```

- Crear el nodo manager

```
docker run -d \
--name manager \
--hostname manager \
--privileged \
--network swarm-network \
-p 3000:3000 \
-p 2377:2377 \
-p 7946:7946 \
-p 7946:7946/udp \
-p 4789:4789/udp \
docker:dind \
dockerd --host=tcp://0.0.0.0:2375 --host=unix:///var/run/docker.sock
```

- Crear nodos

```
docker run -d \
--name worker-1 \
--hostname worker-1 \
--privileged \
--network swarm-network \
docker:dind
```

- Iniciar swarm

```
docker exec manager docker swarm init
```

- compose.yaml

```
name: swarm-cluster

services:
  manager:
    image: docker:dind
    container_name: manager
    hostname: manager
    privileged: true
    networks:
      - swarm-network
    ports:
      - "3000:3000"
      - "2377:2377"
      - "7946:7946"
      - "7946:7946/udp"
      - "4789:4789/udp"
    command: dockerd --host=tcp://0.0.0.0:2375 --host=unix:///var/run/docker.sock

  worker-1:
    image: docker:dind
    container_name: worker-1
    hostname: worker-1
    privileged: true
    networks:
      - swarm-network

  worker-2:
    image: docker:dind
    container_name: worker-2
    hostname: worker-2
    privileged: true
    networks:
      - swarm-network

networks:
  swarm-network:
    driver: bridge
```


Añadir nodos **worker**

Para obtener el comando de unión, incluyendo el token para nodos worker

```
manager: docker swarm join-token worker
```

El comando **swarm join** une los nodos al swarm y realiza las siguientes acciones:

- Extiende la red overlay
- Solicita un certificado TLS

```
worker-1: docker swarm join ....  
worker-2: docker swarm join ....
```

Añadir nodos **manager**

Para obtener el comando de unión, incluyendo el token para nodos manager:

```
manager: docker swarm join-token manager
```

```
manager-2: docker swarm join ....
```

Gestionar nodos

Para ver todos los nodos que forman parte del swarm, ejecuta el siguiente comando desde un nodo manager:

```
manager: docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
o5zoqjxhgyfn14qk2w1mpy1xu *	manager	Ready	Active	Leader	28.5.0
ml6v08wcxlcqvagmjg69pj27m	worker-1	Ready	Active		28.5.0
y9haoji1t9316n6g9m03au2im	worker-2	Ready	Active		28.5.0

- **AVAILABILITY:** Indica si el planificador puede asignar tareas al nodo:
 - **Active:** El nodo puede recibir tareas nuevas.
 - **Pause:** No recibe tareas nuevas, pero las existentes siguen ejecutándose.
 - **Drain:** No recibe tareas nuevas y las existentes se reubican en otros nodos.

- **MANAGER STATUS:** Muestra la participación del nodo en el consenso Raft:
 - *(Vacío)*: Nodo worker, no participa en la gestión del swarm.
 - **Leader**: Nodo manager principal, toma decisiones de orquestación.
 - **Reachable**: Manager participante en el quorum Raft, elegible como nuevo líder si el actual falla.
 - **Unavailable**: Manager que no puede comunicarse con otros managers; es recomendable añadir o promover otro manager.

Cambiar Estado

Cambiar el parámetro availability de un nodo en Docker Swarm sirve para controlar si ese nodo puede recibir nuevas tareas o no.

```
manager: docker node update --availability pause|drain|active worker-1
```

```
manager: docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER	STATUS	ENGINE	VERSION
jijwfjp1atvi3ho32puxax1pe *	silvelo	Ready	Active	Leader		28.5.0	
gi7u04ihrvr95dn6ei1144vyh	worker-1	Ready	Pause			28.5.0	
l5lddhqxecbkhq1ue578z9c6x	worker-2	Ready	Active			28.5.0	

Promover o degradar un nodo

Puedes **promover** un nodo worker a manager, lo cual es útil si un manager queda fuera de servicio o si necesitas más managers para mantener el quorum.

```
manager: docker node promote worker-1 worker-2
```

```
manager: docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
jijwfjp1atvi3ho32puxax1pe *	silvelo	Ready	Active	Leader	28.5.0
gi7u04ihrvr95dn6ei1144vyh	worker-1	Ready	Active	Reachable	28.5.0
l5lddhqxecbkhq1ue578z9c6x	worker-2	Ready	Active	Reachable	28.5.0

De igual forma, puedes **degradar** (demote) un manager a worker, por ejemplo, para realizar tareas de mantenimiento o reducir el número de managers.

```
manager: docker node demote worker-1 worker-2
```

```
manager: docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
jijwfjp1atvi3ho32puxax1pe *	silvelo	Ready	Active	Leader	28.5.0
gi7u04ihrvr95dn6ei1144vyh	worker-1	Ready	Active		28.5.0
l5lddhqxecbkhq1ue578z9c6x	worker-2	Ready	Active		28.5.0

Eliminar nodo

En ocasiones es necesario eliminar un nodo del swarm, por ejemplo, cuando un nodo deja de estar disponible, se va a dar de baja definitivamente o simplemente quieres reorganizar el cluster.

- Nodo abandone el swarm

```
worker-2: docker swarm leave
```

- Eliminar nodo del swarm

```
manager: docker node rm worker-2
```

Para nodos manager primero hay que degradar el nodo

Servicios

Crear

Para desplegar una aplicación en el cluster Swarm, se utiliza el concepto de **servicio**. Un servicio define la imagen del contenedor, el número de réplicas y otras opciones de despliegue.

```
manager: docker service create --name single ghcr.io/trynewroads/docker-course-advance:latest
```

Listar servicios y contenedores

Para gestionar y supervisar el estado de las aplicaciones desplegadas en un clúster Swarm, es fundamental listar los servicios activos:

```
manager: docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
prronr507803	single	replicated	2/2	ghcr.io/trynewroads/docker-course-advance:latest	*:3000->3000/tcp

y los contenedores (tareas) que los componen.

```
manager: docker service ps single
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
prronr507803	single	replicated	1/1	ghcr.io/trynewroads/docker-course-advance:latest	*:3000->3000/tcp

Escalar

Puedes aumentar o disminuir el número de réplicas de un servicio en Swarm de forma sencilla usando el comando `docker service scale`.

```
docker service scale single=5
```

Esto ajusta el servicio `single` para que tenga 5 réplicas activas distribuidas entre los nodos del cluster.

Actualizar

Puedes modificar casi cualquier aspecto de un servicio existente usando el comando `docker service update`. Al actualizar un servicio, Docker detiene sus contenedores y los reinicia con la nueva configuración.

```
manager: docker service update --publish-add 3000:3000 --constraint-add 'node.role==worker' single
```

Para eliminar la configuración:

```
manager: docker service update --publish-rm 3000 --constraint-rm 'node.role==worker' single
```

Rolling update

Actualización progresiva de las tareas de un servicio para minimizar downtime y permitir revertir si algo falla.

- **--update-parallelism**: cuántas tareas se actualizan - simultáneamente.
- **--update-delay**: espera entre batches.
- **--update-monitor**: tiempo que Docker observa la tarea tras el update.
- **--update-failure-action**: acción si falla (continue | rollback | pause).

- Creamos un servicio con múltiples réplicas:

```
docker service create --name web \  
  --replicas 6 \  
  --publish 3000:3000 \  
  ghcr.io/trynewroads/docker-course-advance:1.5
```

- Actualizamos la imagen del servicio:

```
docker service update \  
  --image ghcr.io/trynewroads/docker-course-advance:1.6 \  
  --update-parallelism 2 \  
  --update-delay 10s \  
  --update-monitor 30s \  
  --update-failure-action rollback \  
  web
```


Acción de revertir una actualización de un servicio a la última especificación conocida y funcional (imagen, variables, mounts y configuración de deploy).

Cuando sucede un rollback:

- Si la actualización detecta fallos durante la ventana de monitorización.

```
--update-failure-action rollback
```

- Ejecución manual

```
docker service rollback
```

Es útil para recuperar rápidamente un estado estable cuando un rolling update introduce contenedores unhealthy o errores.

Opciones Rollback

Para controlar el ritmo de actualización de un servicio podemos modificar la configuración del servicio con:

- **--rollback-parallelism N**: Cuántas tareas se revierten simultáneamente.
- **--rollback-delay DURATION**: Espera entre batches durante el rollback.

Comprobar Servicio

Comprobar el estado, la configuración y la historia de un servicio para entender cómo se está comportando:

- Toda la configuración

```
docker service inspect --pretty web
```

- Estado anterior

```
docker service inspect --format '{{json .PreviousSpec}}' web
```

- Estado de la última actualización

```
docker service inspect --format '{{json .UpdateStatus}}' web
```

- Comprobar la configuración del rollback

```
docker service inspect --format '{{json .Spec.RollbackConfig}}' web
```

- Cambiar la configuración

```
docker service update --rollback-parallelism 3 --rollback-delay 10s web
```

- Hacer el rollback

```
docker service rollback web
```

HEALTHCHECK y --update-monitor

Swarm usa los HEALTHCHECK de la imagen para decidir si una tarea nueva está healthy.

Si --update-monitor es menor que la ventana necesaria para que el HEALTHCHECK marque "unhealthy", Swarm no detectará el fallo y la actualización seguirá sin pausar ni revertir.

```
monitor >= StartPeriod + Interval * Retries
```

- Rollback del servicio en caso de error.

```
docker service update \  
  --image ghcr.io/trynewroads/docker-course-advance:fail \  
  --update-parallelism 3 \  
  --update-delay 10s \  
  --update-monitor 40s \  
  --update-failure-action rollback \  
web
```

- Inspeccionamos el servicio (6/6 Replicas)

```
docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
xa2s84kn9otz	web	replicated	6/6	ghcr.io/trynewroads/docker-course-advance:1.6	

- Pausar la actualización del servicio en caso de error.

```
docker service update \  
  --image ghcr.io/trynewroads/docker-course-advance:fail \  
  --update-parallelism 3 \  
  --update-delay 10s \  
  --update-monitor 40s \  
  --update-failure-action pause \  
web
```

- Inspeccionamos el servicio (3/6 Replicas)

```
docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
xa2s84kn9otz	web	replicated	3/6	ghcr.io/trynewroads/docker-course-advance:fail	

- Continuar la actualización del servicio en caso de error.

```
docker service update \  
  --image ghcr.io/trynewroads/docker-course-advance:fail \  
  --update-parallelism 3 \  
  --update-delay 10s \  
  --update-monitor 40s \  
  --update-failure-action continue \  
web
```

- Inspeccionamos el servicio (0/6 Replicas)

```
docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
xa2s84kn9otz	web	replicated	0/6	ghcr.io/trynewroads/docker-course-advance:fail	

Conexiones y balanceo en Docker Swarm

Docker Swarm gestiona automáticamente el enrutamiento y balanceo de las conexiones hacia los servicios desplegados en el clúster. Cuando un servicio está publicado en un puerto, cualquier petición enviada a ese puerto en cualquier nodo del clúster será redirigida de forma transparente a una de las réplicas disponibles del servicio.

En el caso de nuestra aplicación, si accedemos `http://localhost:3000/`, el servidor responde con un JSON que incluye la propiedad `hostname`, la cual corresponde al identificador del contenedor que ha gestionado la petición.

Redes en Docker Swarm

En Docker Swarm, la gestión de redes es fundamental para la comunicación y el aislamiento de los servicios desplegados en el clúster.

- **Redes overlay:**

Swarm introduce el concepto de redes overlay, que permiten la comunicación entre contenedores distribuidos en diferentes nodos del clúster. Estas redes se crean a nivel de Swarm y conectan servicios de forma segura y transparente, independientemente del nodo en el que se encuentren.

- **Red ingress:**

Así como Docker clásico utiliza la red bridge para conectar contenedores en un solo host, Swarm utiliza la red ingress como su análogo.

- **Aislamiento de servicios:**

Al igual que en Docker tradicional se pueden aislar contenedores usando diferentes redes bridge, en Swarm es posible aislar servicios utilizando distintas redes overlay.

- Crear una red overlay

```
manager: docker network create --driver overlay my-overlay
```

- Añadir red al servicio

```
manage: docker service update --network-add my-overlay single
```

- Comprobar la red

```
manager: docker network inspect my-overlay  
worker-1: docker network inspect my-overlay
```

Volume

En Docker Swarm, los volúmenes se gestionan a nivel de clúster y permiten almacenar datos persistentes para los servicios.

Puedes configurar un volumen en Docker Swarm usando la opción `--mount` al crear un servicio, o bien con `--mount-add` y `--mount-rm` para añadir o eliminar volúmenes en servicios ya existentes.

Los volúmenes pueden crearse antes de desplegar un servicio, o bien, si no existen en un host cuando una tarea es programada allí, Docker los crea automáticamente según la especificación del volumen en el servicio.

- Añadir el volume

```
manager: docker service update --mount-add type=volume,source=app-uploads,target=/app/uploads single
```

- Verificar la creación del volume

```
manager: docker volume ls  
worker-1: docker volume ls  
worker-2: docker volume ls
```

- Subir un archivo

```
curl -X POST http://localhost:3000/upload -F "file=@<file-path>" -H "Content-Type: multipart/form-data"
```

- Verificar cada contenedor: El contenido del volumen es diferente en cada contenedor porque, por defecto, los volúmenes se crean de manera local en cada nodo donde se ejecuta una tarea del servicio.

Eliminar

Para eliminar un servicio en Docker Swarm y detener todas sus tareas y contenedores asociados.

```
docker service rm single
```

Esto elimina el servicio `single` y libera los recursos en todos los nodos del cluster.

Stack

Un stack es una colección de servicios que se despliegan y gestionan juntos como una sola unidad. Se define mediante un archivo YAML que describe todos los componentes y configuraciones de tu aplicación, incluyendo servicios, redes y volúmenes.

Cuando despliegas un stack, Docker Swarm crea todos los servicios definidos en ese archivo y los agrupa bajo un mismo nombre de stack, facilitando la gestión y el despliegue de aplicaciones complejas con múltiples servicios relacionados

Deploy

El bloque `deploy` en un servicio de Docker Swarm permite definir políticas y restricciones de despliegue avanzadas. Los elementos principales son:

- **replicas:** Número de instancias (réplicas) del servicio que se desean ejecutar.
- **placement:** Restricciones para decidir en qué nodos se ejecuta el servicio (por ejemplo, solo en managers o solo en workers).

- **resources:** Límites y reservas de recursos (CPU y memoria) para cada contenedor del servicio.
- **restart_policy:** Política de reinicio automático de los contenedores en caso de fallo.
- **rollback_config:** Permite definir cómo se comporta el servicio durante un rollback (reversión) tras un fallo en una actualización.
- **update_config:** Permite definir cómo se realiza la actualización de un servicio en Swarm.

- stack.yaml

```
services:
  app-base:
    image: ghcr.io/trynewroads/docker-course-advance:latest
    environment:
      USE_DB: "true"
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASS: password
      DB_NAME: postgres
    ports:
      - "3005:3000"
    deploy:
      replicas: 2
      resources:
        limits:
          cpus: "0.50"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 256M
      placement:
        constraints:
          - node.role == worker

  postgres:
    image: postgres:15
    deploy:
      placement:
        constraints:
          - node.role == manager
      replicas: 1
      resources:
        limits:
          cpus: "1.00"
          memory: 1G
        reservations:
          cpus: "0.50"
          memory: 512M
    environment:
      POSTGRES_PASSWORD: password

networks:
  default:
    driver: overlay
    name: app-base
```

- Iniciamos el stack

```
manager:docker stack deploy -c stack.yaml app
```

- Comprobamos el stack

```
manager:docker stack services app
```

- Añadimos usuarios

```
curl -X POST http://localhost:3001/users -H "Content-Type: application/json" -d '{"name":"Juan Pérez","email":"juan@example.com"}'
curl -X POST http://localhost:3001/users -H "Content-Type: application/json" -d '{"name":"Juan Pérez","email":"juan@example.com"}'
curl -X POST http://localhost:3001/users -H "Content-Type: application/json" -d '{"name":"Juan Pérez","email":"juan@example.com"}'
```

```
curl http://localhost:3001/users | jq '.data[] | {name, email}'
```

- Eliminamos el stack

```
manager:docker stack rm app
```

Secretos

Secretos

En Docker Swarm, un **secreto** es un fragmento de información sensible, como una contraseña, clave privada SSH, certificado SSL u otro dato que no debe transmitirse por la red ni almacenarse sin cifrar en un Dockerfile o en el código fuente de la aplicación.

Docker Swarm permite gestionar estos secretos de forma centralizada y segura, transmitiéndolos únicamente a los contenedores que los necesitan. Los secretos están cifrados tanto en tránsito como en reposo dentro del clúster Swarm.

Esto garantiza que la información sensible esté protegida y solo sea accesible por los servicios autorizados durante su ejecución.

Crear

Para crear un secreto en Docker Swarm, utiliza el siguiente comando desde un nodo manager:

```
manager: echo 'password' > db_password_text  
manager: docker secret create db_password_text db_password_text  
manager: rm db_password_text
```

```
manager: echo "password" | docker secret create db_password -
```

Listar

Para ver todos los secretos almacenados en el clúster Swarm:

```
manager: docker secret ls
```

ID	NAME	DRIVER	CREATED	UPDATED
ta8leq2w3g5spm8earmgpre8l	db_password_text		3 seconds ago	3 seconds ago
srponfu13xcdghfe8c2on30t8	db_password		About a minute ago	About a minute ago

Cómo usar secretos

Para utilizar secretos en Docker Swarm, debes tener en cuenta que **los secretos no se exponen como variables de entorno**, sino como archivos de solo lectura dentro del contenedor, ubicados en `/run/secrets/<nombre_secreto>`.

Por tanto, tu aplicación debe estar preparada para **leer el contenido de estos archivos de texto** para obtener, por ejemplo, contraseñas o claves privadas.

- Iniciar servicio con secret

```
manager: echo "my_super_secret" | docker secret create super_secret -
```

```
manager: docker service create \
  --name app-secret \
  --publish 3004:3000 \
  --secret source=super_secret,target=secret \
  ghcr.io/trynewroads/docker-course-advance:latest
```

- Verificar

```
curl http://localhost:3004/secret
{
  "secret": "my_super_secret",
  "timestamp": "2025-10-08T22:37:09.488Z"
}
```

- compose.yaml: Secret externo creado por cli

```
services:
  app-base:
    hostname: app-base
    image: ghcr.io/trynewroads/docker-course-advance:latest
    environment:
      DEBUG_LEVEL: debug
      USE_DB: "true"
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_NAME: postgres
    secrets:
      - db_password
    ports:
      - "3005:3000"
    deploy:
      replicas: 2
      placement:
        constraints:
          - node.role == worker

  postgres:
    hostname: postgres
    image: postgres:15
    deploy:
      placement:
        constraints:
          - node.role == manager
      replicas: 1
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    external: true

networks:
  default:
    driver: overlay
```

- Ejecución

```
manager: docker stack deploy -c stack-secret.yaml m2
```

- Verificación

```
manager: docker stack ls
manager: docker stack ps m2
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
xgk2g1m21v1m	m2_app-base.1	ghcr.io/trynewroads/docker-course-advance:latest	worker-2	Running	Running 3 minutes ago		
rres1u8letk	m2_app-base.1	ghcr.io/trynewroads/docker-course-advance:latest	worker-2	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
n1piy9kuaiis	m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Running	Running 3 minutes ago		
va3pm957srjq	m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
zq6paqrsna8r	m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
4yqn7x9az54k	m2_postgres.1	postgres:15	manager	Running	Running 3 minutes ago		

- Comprobación secreto

```
manager: docker service ls
manager: docker service logs m2_app-base
```

Como los servicios se ejecutan en paralelo podemos encontrar problemas de que `postgres` no esta inicializado y el `backend` si, y este no pueda comunicarse con él, que es lo que se observa `docker stack ps m2`, que algunos contenedores han finalizado por problemas de conexión.

- compose.yaml: Secret externo creado por fichero

```
services:
  app-base:
    hostname: app-base
    image: ghcr.io/trynewroads/docker-course-advance:latest
    environment:
      DEBUG_LEVEL: debug
      USE_DB: "true"
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_NAME: postgres
    ports:
      - "3005:3000"
    secrets:
      - source: db_password_text
        target: db_password
    deploy:
      replicas: 2
      placement:
        constraints:
          - node.role == worker

  postgres:
    hostname: postgres
    image: postgres:15
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == manager
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password_text
    secrets:
      - db_password_text

secrets:
  db_password_text:
    external: true

networks:
  default:
    driver: overlay
    name: app-base
```

- Ejecución

```
manager: docker stack deploy -c stack-secret.yaml m2
```

- Verificación

```
manager: docker stack ls
manager: docker stack ps m2
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
xgk2g1m2ivim	m2_app-base.1	ghcr.io/trynewroads/docker-course-advance:latest	worker-2	Running	Running 3 minutes ago		
rred1u8letk	_ m2_app-base.1	ghcr.io/trynewroads/docker-course-advance:latest	worker-2	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
nlp1y9kuaais	m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Running	Running 3 minutes ago		
va3pm957srjq	_ m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
zq6paqrsna8r	_ m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
4yqn7x9az5ak	m2_postgres.1	postgres:15	manager	Running	Running 3 minutes ago		

- Comprobación secreto

```
manager: docker service ls
manager: docker service logs m2_app-base
```

- compose.yaml: Secret propio del fichero

```
services:
  app-base:
    hostname: app-base
    image: ghcr.io/trynewroads/docker-course-advance:latest
    environment:
      DEBUG_LEVEL: debug
      USE_DB: "true"
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_NAME: postgres
    secrets:
      - db_password
    ports:
      - "3005:3000"
    deploy:
      replicas: 2
      placement:
        constraints:
          - node.role == worker

  postgres:
    hostname: postgres
    image: postgres:15
    deploy:
      placement:
        constraints:
          - node.role == manager
      replicas: 1
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    file: ./db_password.txt

networks:
  default:
    driver: overlay
```

- Ejecución

```
manager: docker stack deploy -c stack-secret.yaml m2
```

- Verificación

```
manager: docker stack ls
manager: docker stack ps m2
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
xgk2g1m21v1m	m2_app-base.1	ghcr.io/trynewroads/docker-course-advance:latest	worker-2	Running	Running 3 minutes ago		
rre6m1u8letk	_ m2_app-base.1	ghcr.io/trynewroads/docker-course-advance:latest	worker-2	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
nlpiy9kuaiis	m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Running	Running 3 minutes ago		
va3pm957srjq	_ m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
zq6paqrsna8r	_ m2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Shutdown	Failed 3 minutes ago	"task: non-zero exit (1)"	
4yqn7x9az54k	m2_postgres.1	postgres:15	manager	Running	Running 3 minutes ago		

- Comprobación secreto

```
manager: docker service ls
manager: docker service logs m2_app-base
```

Eliminar

Para eliminar un secreto del swarm, este debe estar siendo utilizado por ningún servicio activo. Docker Swarm no permite borrar secretos que estén en uso.

```
docker secret rm my_super_secreet
```

Dependencias

En Docker Swarm, el atributo `depends_on` de Docker Compose no tiene soporte completo como en Compose tradicional: solo garantiza el orden de creación de los servicios, pero no espera a que los servicios dependientes estén saludables antes de iniciar el siguiente servicio.

- La gestión de dependencias debe manejarse dentro de la propia aplicación
- Mediante scripts de espera en el entrypoint del contenedor

```
services:
  app-base:
    hostname: app-base
    image: ghcr.io/trynewroads/docker-course-advance:latest
    entrypoint: ["/bin/sh", "-c"]
    command: >
      'until nc -zv postgres 5432; do echo "waiting for postgres"; sleep 2; done; node /app/src/app.js'
    environment:
      DEBUG_LEVEL: debug
      USE_DB: "true"
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_NAME: postgres
    secrets:
      - db_password
    ports:
      - "3005:3000"
    deploy:
      replicas: 2
      placement:
        constraints:
          - node.role == worker

  postgres:
    hostname: postgres
    image: postgres:15
    deploy:
      placement:
        constraints:
          - node.role == manager
      replicas: 1
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    external: true

networks:
  default:
    driver: overlay
```

• Ejecución

```
manager: docker stack deploy -c stack-secret.yaml d2
```

• Verificación

```
manager: docker stack ls
docker stack ps d2
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
w8ldkzybjatu	d2_app-base.1	ghcr.io/trynewroads/docker-course-advance:latest	worker-2	Running	Running about a minute ago		
m8x6zbjz3xfz	d2_app-base.2	ghcr.io/trynewroads/docker-course-advance:latest	worker-1	Running	Running about a minute ago		
sqcfcaw54hg	d2_postgres.1	postgres:15	manager	Running	Running 2 minutes ago		

• Comprobación secreto

```
manager: docker service ls
manager: docker service logs d2_app-base
```